

OpenMP (Open Multi-Processing)

- API (application programming interface) (интерфейс прикладного программирования), предназначенный для распараллеливания программ
- Изначально для систем с общей памятью (в последних версиях стандарта возможны иные варианты)
- Кросс-платформенный
- Основные реализации – изначально для языков C/C++ и Fortran
- Сосуществование, взаимодействие и конкуренция стандартов - де-факто сосуществование с MPI на данный момент, перспективы – имеем возможность наблюдать за развитием, пересечение возможностей с библиотеками и средствами C++, OpenCL и др.

Поддержка стандарта компиляторами

- gcc,
- LLVM (Clang),
- Microsoft Visual C++,
- Intel(C++ compiler, Fortran, Parallel Studio), AMD (AOCC), ARM,
- XL C/C++ (IBM), Oracle (ex. Sun Studio в том числе), PGI, Absoft Pro Fortran, Lahey/Fujitsu Fortran, PathScale, Cray, NAG, OpenUH Research Compiler, LLNL Rose Research Compiler, Appentra Solutions parallware compiler, Texas Instruments (C), Barcelona Supercomputing Center, HPE CCE, NVIDIA HPC Compiler, Oracle Developer Studio, Siemens.... и другие.
- Также есть поддержка рядом утилит (профилировщиков, санитайзеров кода и т.д.) (Codee, gprofng, VTune Profiler, Advisor , Inspector и рядом других)

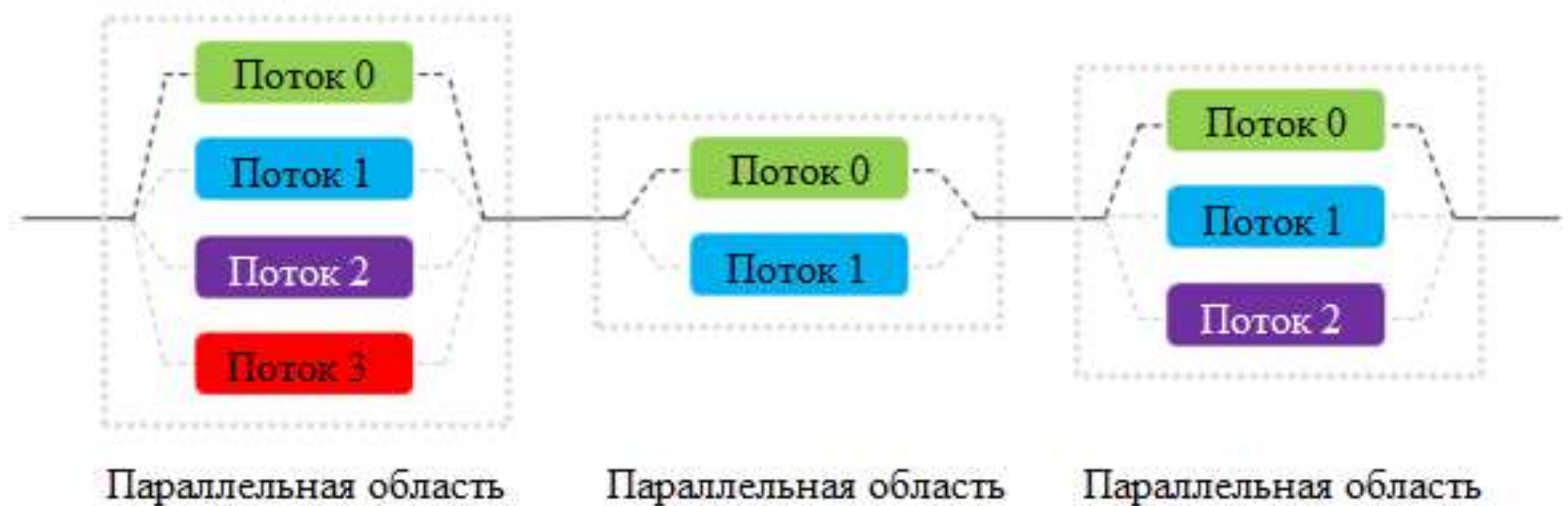
Что входит в стандарт?

- Для пользователя предоставляются:
- - директивы компилятора (`#pragma` в C)
- - библиотечные функции (процедуры)
- - переменные окружения

Изначально – для PRAM (parallel random-access machine)
(SMP - symmetric multiprocessing и т.д. – подвиды)

(дополнительное задание – как работают директивы компилятора)

«Классическая» модель OMP

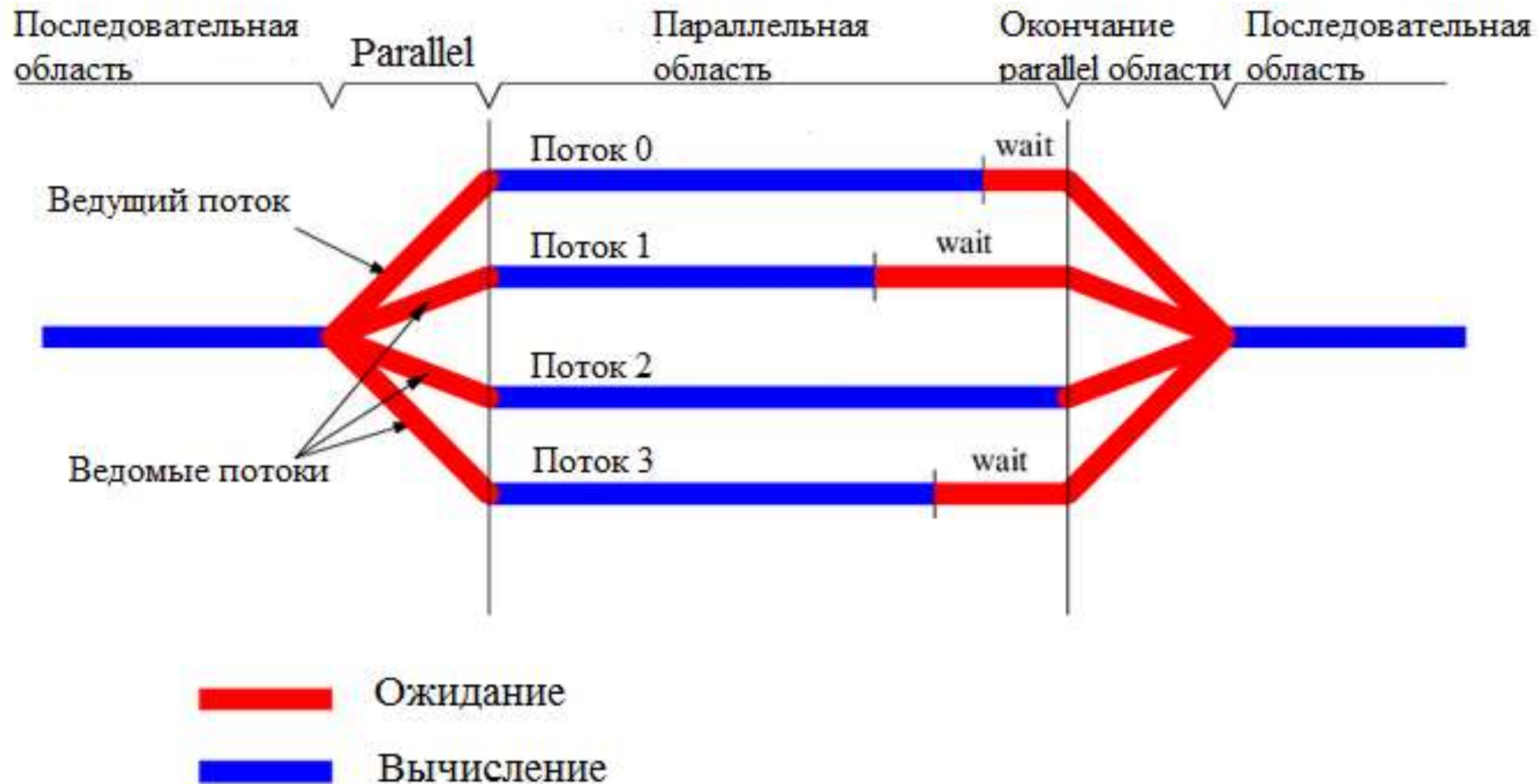


Первые шаги

- `#include <omp.h>` - подключение библиотеки (для языка C)
- Опция компилятора (gcc): **gcc -fopenmp** (и остальные опции, если требуются)
- Все директивы OpenMP начинаются с **#pragma omp**
- Также имеется набор функций
- Одна из директив – **parallel**

```
#pragma omp parallel
{
    // Код внутри блока выполняется параллельно.
    printf("Hello! \n");
}
```

Директива parallel



Методы обеспечения совместимости (для языка C)

- Если компилятор не поддерживает `openmp`, директивы `#pragma omp` игнорируются
- Макрос `_OPENMP` (показывает наличие поддержки `OPENMP` и версию в формате `уууутт`, где `уууу` – год, а `тт` – месяц принятия соответствующей версии стандарта
- Директива `if`

Примеры кода

```
#ifdef _OPENMP
```

```
    int max_threads = omp_get_max_threads();
```

```
    //(201307 – версия openmp 4.0)
```

```
#if _OPENMP >= 201307
```

```
#pragma omp simd
```

```
    for (int i = 0; i < 16; ++i) //simd рассмотрим позже
```

```
        data[i] += x[i];
```

```
#else
```

```
    for (int i = 0; i < 16; ++i)
```

```
        data[i] += x[i];
```

```
#endif
```


Примеры кода - 2

```
#include <omp.h>
#include <stdio.h>
int main (void) {
    int t = 1;
    int f = 0;
    #pragma omp parallel if (f)
        printf ("FALSE: I am thread %d\n", omp_get_thread_num());
    #pragma omp parallel if (t)
        printf ("TRUE : I am thread %d\n", omp_get_thread_num());
    return 0;
}
```

***** запуск на 4 потоках

```
FALSE: I am thread 0
TRUE : I am thread 0
TRUE : I am thread 1
TRUE : I am thread 3
TRUE : I am thread 2
```

Число потоков

- Глобальная переменная `OMP_NUM_THREADS`
(`bash# export OMP_NUM_THREADS=16`)
- Функция `void omp_set_num_threads(int num)`
Задаёт число потоков в предстоящих параллельных областях, т.е. по сути переопределяет глобальную переменную в рамках программы, пример вызова `omp_set_num_threads(16);`
- Директива `num_threads(num)`
Задаёт число потоков соответствующей параллельной области
`#pragma omp parallel num_threads(8)`
{что-то, что выполнится при помощи 8-ми потоков}

Число потоков – 2

- `int omp_get_max_threads(void)`

сколько всего можно создать потоков

- `int omp_get_num_procs(void)`

сколько всего процессорных ядер?

//чаще всего для вычислительных задач больше потоков создавать нет смысла)

- `int omp_get_num_threads(void)`

сколько всего потоков в параллельной области

- `int omp_get_thread_num(void)`

узнать свой номер – можно использовать для построения алгоритмов

Число потоков - дополнительно

- Управление динамической настройкой числа исполнителей в процессе выполнения программы - переменная среды OMP_DYNAMIC, функции `void omp_set_dynamic(int num)` и `int omp_get_dynamic(void)`
- Узнать, в параллельной ли мы области? Функция `int omp_in_parallel(void)`
- Вложенный параллелизм и его максимальный уровень – переменные среды OMP_NESTED и OMP_MAX_ACTIVE_LEVELS, функции `void omp_set_nested(int nested)` , `int omp_get_nested(void)`, `int omp_get_team_size(int level)` и другие

Программа создана. Измерим производительность?

- `double omp_get_wtime(void)`
//имеет смысл только разница между двумя значениями, абсолютное значение зависит от реализации, значения в разных потоках не синхронизированы
- `double omp_get_wtick(void)`
узнаём разрешение таймера

Распараллеливание циклов

- Директива for

```
#pragma omp for
```

```
    for (i = 0; i < 128; i++)
```

(внутри параллельной области)

```
#pragma omp parallel for
```

```
    for (i = 0; i < 128; i++)
```

//распараллеливается по индексу, цикл проходится один раз

//нет проверки информационных зависимостей!

//без директивы for будет полная неразбериха, так как переменная i – общая, если объявить её в теле цикла – цикл выполнится столько раз, сколько есть потоков

Требования к циклу for

- Зависят от версии стандарта (чем старше стандарт, тем жёстче)
- Не должно быть дополнительных точек выхода вроде `break`
- Во всех версиях работает целочисленным итератором, фиксированным инкрементом/декрементом, заранее определённым значением интервала

Как управлять распараллеливанием?

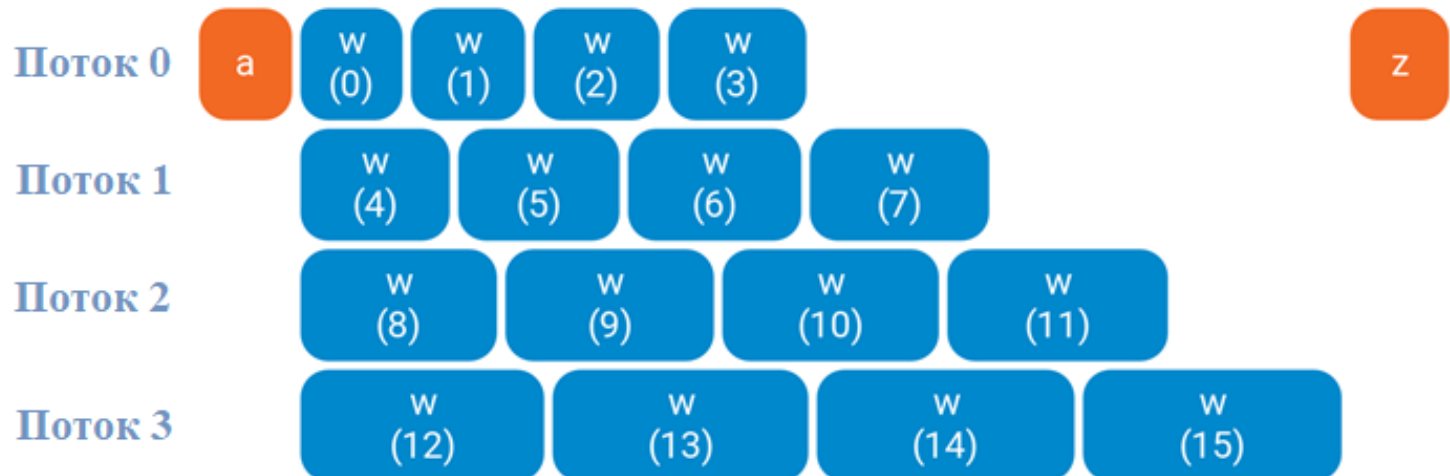
- Есть возможность управления балансировкой (**scheduling**)

Без директивы

a();

```
#pragma omp parallel for  
  for (int i = 0; i < 16; ++i)  
    { w(i); }
```

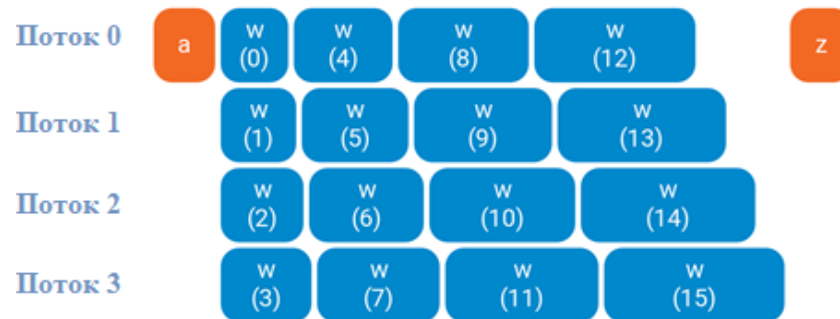
z();



Директива (static, chunk – не обязателен)

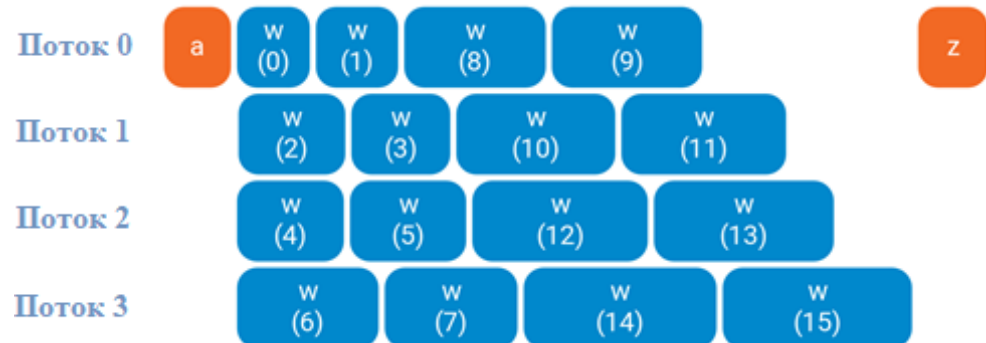
```
a();  
#pragma omp parallel for schedule(static,1)  
    for (int i = 0; i < 16; ++i)  
        { w(i); }
```

z();



```
a();  
#pragma omp parallel for schedule(static,2)  
    for (int i = 0; i < 16; ++i)  
        { w(i); }
```

z();



Директива (dynamic , chunk – не обязателен)

```
#pragma omp parallel for schedule(dynamic)
```

```
    for(int i=0; i<16; i++) printf(" %d \n", i);
```

```
// конвеерный параллелизм, по 1 итерации, аналогично
```

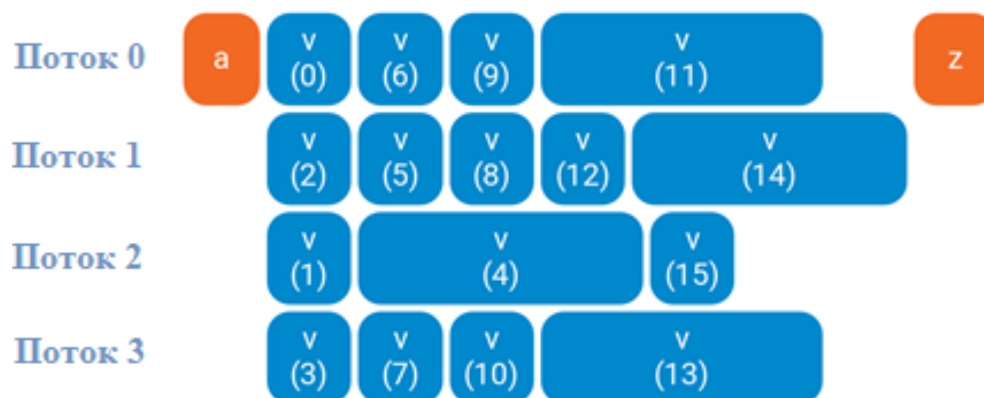
```
a();
```

```
#pragma omp parallel for schedule(dynamic,1)
```

```
    for (int i = 0; i < 16; ++i)
```

```
        {v(i); }
```

```
z();
```



```
#pragma omp parallel for schedule(dynamic,4)
```

```
    for(int i=0; i<16; i++) printf(" %d \n", i);
```

```
// конвеерный параллелизм, по 4 итерации
```

Директивы guided, auto

- guided – используем преимущества dynamic и снижаем накладные расходы за счёт динамически меняющегося шага (от большего к меньшему)
- auto – на выбор системы

//анонс первой задачи со * - на 0.3 балла

//распараллелить цикл на 65 итераций на 4 потоках разными методами балансировки и объяснить результаты

Зоны видимости переменных

- По умолчанию – всё что вне блока – общие, что внутри блока – приватные (т.е. разные для потоков)

```
#pragma omp parallel
```

```
{  
    int num;  
    num = omp_get_thread_num();  
    printf(“%d\n”,num); // у каждого потока переменная num  
                        своя  
}
```

- Можно управлять директивами

Зоны видимости переменных - 2

- `private (список переменных)//через запятую`
(переменные создаются, но не инициализируются)
- `shared (список переменных)`
- `firstprivate (список переменных) //` тут вновь создаваемые переменные инициализируются значением до параллельной области
- `lastprivate (список переменных) //` после выполнения параллельной области в переменную будет записано последнее значение – из последней итерации цикла или из последней секции

Зоны видимости переменных - 3

- `default` `//shared` или `none`, в последнем случае придётся прописывать все переменные в других директивах
- `reduction` (операция:список переменных)

Операции `+`, `-`, `*`, `&`, `|`, `^`, `&&`, `||`, `max`, `min`

(примечание: - работает также как `+`)

Для аддитивных – новые копии переменных устанавливаются в 0. Для мультипликативных – в 1.

- `threadprivate`(список переменных) (для мастер-потока старая переменная, остальным – новая, не инициализированная)
- `copyin`(список переменных) `//разобраться самостоятельно` и написать пример – на 0.2 балла со *

Особняком – итераторы циклов

Дополнение к занятию 2

Повторение – дополнение

OMP_SCHEDULE (пример - "guided,4" или "dynamic")

omp_set_schedule()

omp_get_schedule()

Существует понятие совместимости(примечания:DO есть только в языке Fortran, таблица для старой версии стандарта для примера)

Clause	Directive					
	PARALLEL	DO/for	SECTIONS	SINGLE	PARALLEL DO/for	PARALLEL SECTIONS
IF						
PRIVATE						
SHARED						
DEFAULT						
FIRSTPRIVATE						
LASTPRIVATE						
REDUCTION						
COPYIN						
SCHEDULE						

Секции

Пример кода:

```
#pragma omp parallel sections
{
    #pragma omp section
    {
        Job1();
    }
    #pragma omp section
    Job2();

    #pragma omp section
    Job3();
}
```


Секции - 2

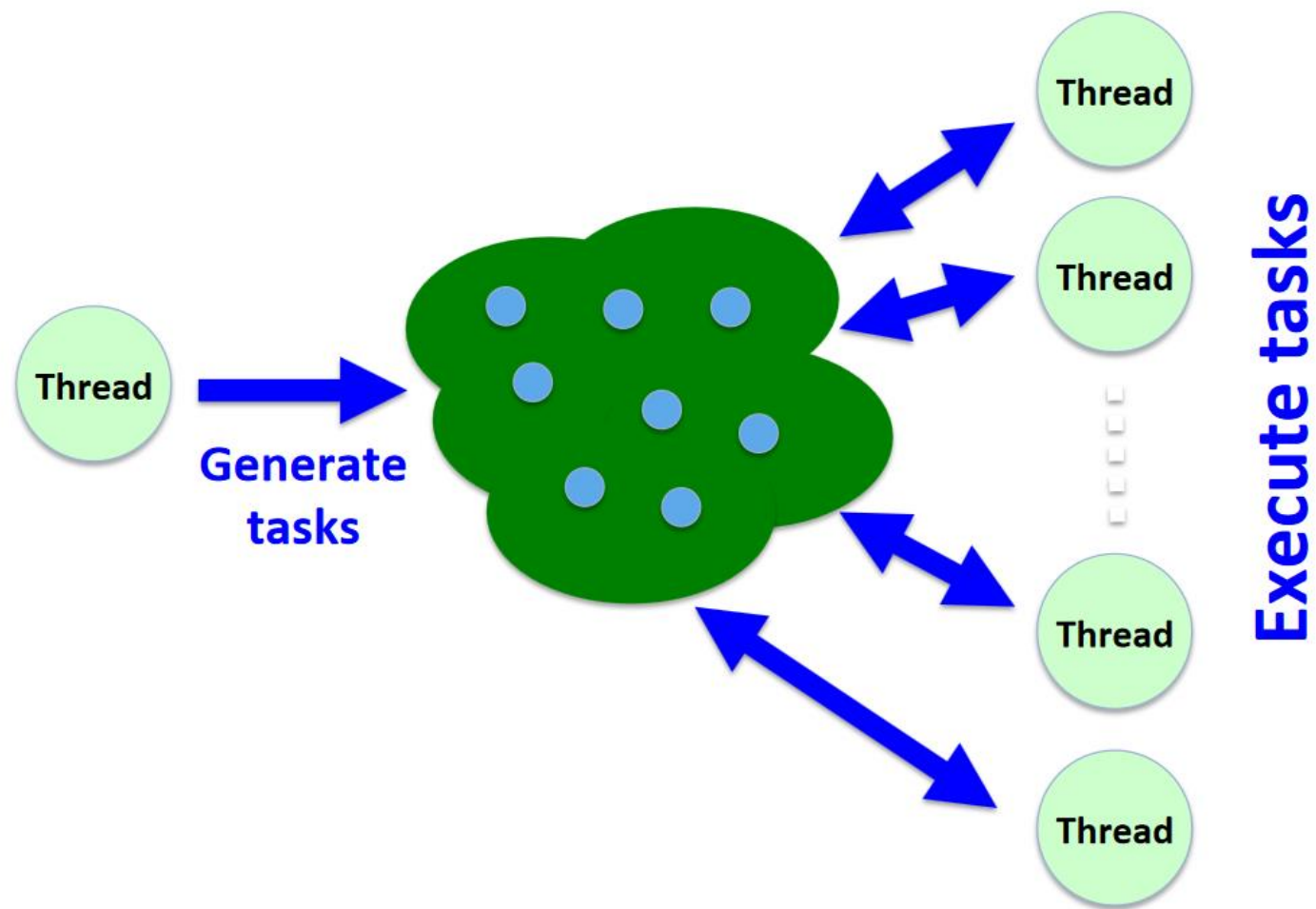
Особенности

1. Синтаксис – требуется директива `sections`, затем перечисляются секции при помощи отдельной директивы `section` на каждую секцию, используется только в параллельной области
2. Гарантируется что каждая секция будет выполнена одним из потоков
3. Балансировка – зависит от реализации, не регламентируется
4. Порядок выполнения не регламентирован
5. Присутствует неявная барьерная синхронизация, если не нужна – директива `nawait`

Задачи (tasks)- основные постулаты

- Блок операторов, который можно выполнить в отдельном потоке, можно вынести в задачу
- При этом новые потоки не создаются, задачи помещаются в специальную область (pool), откуда их может взять любой свободный поток, таким образом производится отход от классической модели «fork/join» с соответствующими накладными расходами
- Появляются возможности для более гибкой работы планировщика и лучшей балансировки ввиду более гибкого распределения потоков между задачами
- Программист должен учитывать условия Бернстайна, чтобы код выполнялся корректно (смотрим лекции за подробностями!)

Задачи (tasks) – схематическое отображение идеи



Задачи (tasks)- особенности реализации

- Начиная с OpenMP 3.0, ряд возможностей – позже
- Поток, встретивший данную конструкцию, может выполнить задачу сразу или поставить её в пул задач
- Задача будет выполнена ровно 1 раз
- Выполнить может не тот поток, который поставил в очередь – а другой из «команды»
- В современных версиях стандарта появилась возможность «привязывать» (tied) или «отвязывать» (untied) задачу, в первом случае она полностью выполняется одним потоком, во втором это не гарантируется (пример – опционально как *)

Задачи (tasks) – синтаксис

- Конструкция task

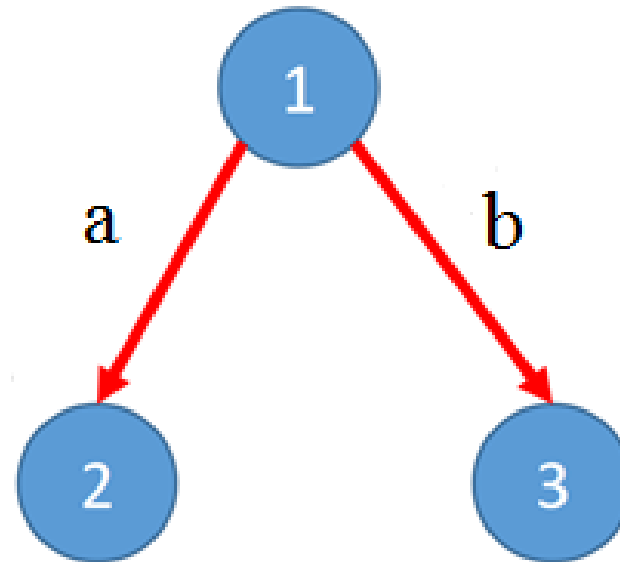
#pragma omp task

Может сочетаться с: //часть только с OpenMP 3.1 или 4.0

- if (выражение) // если выражение не выполняется, выполнение сразу!
- final (выражение)//если выражение, то задание и все его дочерние задачи становятся final и included (т.е. выполняются сразу тем же потоком, разница с if – распространение на иерархию)
- untied //см. предыдущий слайд
- mergeable //может сделаться merged, т.е. с той же средой, что породивший ей процесс
- default (shared | firstprivate | none)
- private (список)
- firstprivate (list)
- shared (список)
- depend (список) // можно явно задавать, от чего зависит выполнение задания)
- priority (значение) // больше – выше приоритет

Пример кода

```
#pragma omp task depend (out a, b)  
task1(...);  
#pragma omp task depend (in a)  
task2(...);  
#pragma omp task depend (in b)  
task3(....);
```



Директива taskwait и другие

- taskwait - ждём выполнения всех задач, порождённых этим потоком (только один уровень)
- taskyield – можно приостановить эту задачу, если есть другие
- taskgroup – как taskwait, только ждём ещё порождённых дочерними задачами задач, без ограничения на уровень вложенности
- taskloop // и многие другие /////пример!

```
printf("Before task");
```

```
#pragma omp task
```

```
printf("Task 1 ");
```

```
#pragma omp task
```

```
printf("Task 2 ");
```

```
#pragma omp taskwait
```

```
//какой-то код после выполнения задачи
```

taskloop

```
#pragma omp parallel
```

```
#pragma omp single
```

```
{
```

```
    #pragma omp taskloop
```

```
    for (int i = 0; i < n; i++)
```

```
        task_implementation(data[i]);
```

```
}
```

```
//каждая итерация цикла станет отдельной задачей
```


Средства синхронизации потоков - 1

- `single` //следующий блок будет выполняться одним потоком, применяется внутри параллельной секции

Пример:

```
#pragma omp single //внутри параллельной области
```

```
#pragma omp task
```

```
{ } // какое-то задание, которое нужно поставить в пул один раз
```

- `master` //следующий блок выполняется мастер-процессом
- `barrier` //явно описанный барьер, часто ещё встречается неявная барьерная синхронизация

Средства синхронизации потоков - 2

- `critical` //обозначает критический блок кода – который может в единицу времени выполнять только 1 поток
- `atomic` //в отличии от критической секции, защищает только одну операцию из ограниченного списка (+, *, -, /, &, ^, |, <<, или >>, не перегруженные (для C++) ++, --, и ряд других конструкций вроде { v = x; x++; })(да, в последних версиях блоки разрешены, но строго из списка), меньше накладные расходы, дополнительные опции `read` | `write` | `update` | `capture`
- `nowait` //очень важная директива, если хочется избавиться от скрытой барьерной синхронизации

```
#pragma omp parallel for
```

```
for (i = 1; i < 128; i++)
```

```
{
```

```
//здесь скрытый барьер
```

nowait

Особенности:

- не совместима с `pragma omp parallel`

Т.е.

`#pragma omp parallel for nowait` = ошибка!

Но!

```
#pragma omp parallel
```

```
{
```

```
#pragma omp for nowait
```

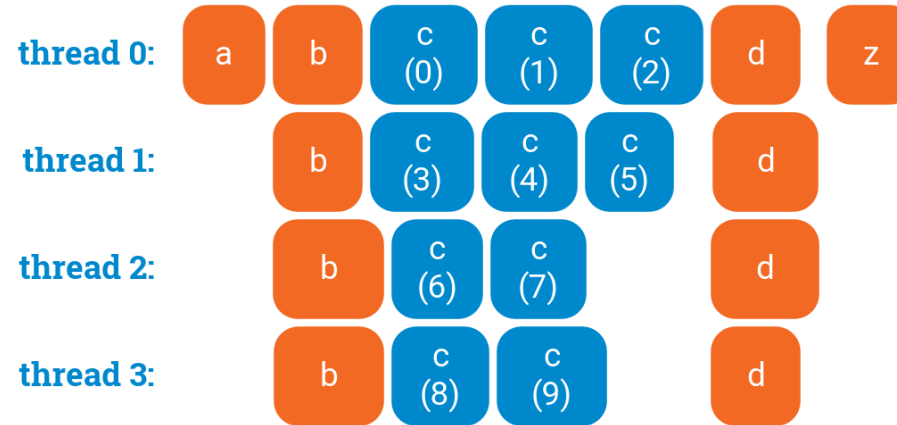
```
for (.....
```

```
} – будет работать
```

//совместимость может зависеть от версии стандарта

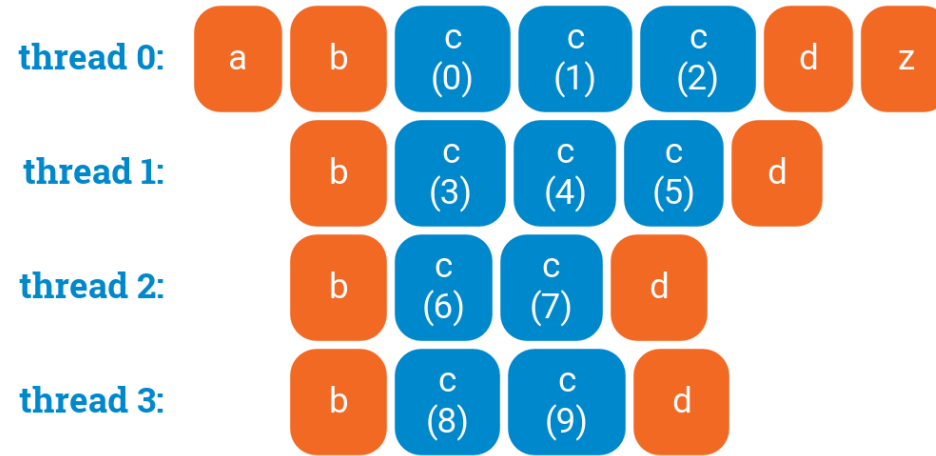
Без nowait

```
a();  
#pragma omp parallel  
{  
    b();  
    #pragma omp for  
    for (int i = 0; i < 10; ++i)  
    {  
        c(i);  
    }  
    d();  
}  
z();
```



C nowait

```
a();  
#pragma omp parallel  
{  
    b();  
    #pragma omp for nowait  
    for (int i = 0; i < 10; ++i)  
    {  
        c(i);  
    }  
    d();  
}  
z();
```



Средства синхронизации потоков - 3

- `ordered` (применяется для циклов, участок кода, который должен выполняться потоками поочерёдно)

//

Последняя «вводная» задача – обеспечить поочерёдный последовательный доступ всех потоков к одной ячейке памяти (например, переменной типа `int`), действие (например, `++`), вывод (номер потока, значение переменной)

- `flush` - точка, в которой должна быть гарантирована синхронизация содержимого всех видов памяти для всех потоков

Блокировки

Тип данных для простых блокировок - `omp_lock_t`, схож с мьютексом

Инициализируется функцией `void omp_init_lock (omp_lock_t *lock)`

После использования - `void omp_destroy_lock(omp_lock_t *lock)`

Используем:

`void omp_set_lock(omp_lock_t *lock)` //блокирующая попытка
занять `lock`, если кто-то занял до этого потока – ждём

`void omp_unset_lock(omp_lock_t *lock)` //при первой возможности

`int omp_test_lock(omp_lock_t *lock)` //неблокирующая попытка
занять `//lock`

Также есть `omp_nest_lock_t`, `omp_init_nest_lock()` и т.д., схож с семафором

Пример кода

```
omp_lock_t simple_lock;
int main() {
    omp_init_lock(&simple_lock);
#pragma omp parallel num_threads(4)
    {
        int tid = omp_get_thread_num();
        while (!omp_test_lock(&simple_lock))
            printf("Thread %d - failed to acquire simple_lock\n", tid);
        printf("Thread %d - acquired simple_lock\n", tid);
        printf("Thread %d - released simple_lock\n", tid);
        omp_unset_lock(&simple_lock);
    }
    omp_destroy_lock(&simple_lock);
}
```


Возможности новых версий OpenMP

Обзор самого интересного

Предупреждение – описанные возможности есть в стандарте, но далеко не всегда реализованы в конкретной версии компилятора. В частности, на учебном сервере большинство указанных в данной презентации конструкций не работает

Куда движется стандарт и кто его развивает?

OpenMP Architecture Review Board (OpenMP ARB)

Текущие приоритеты – многоязыковой параллелизм высокого уровня, производительность, продуктивность и переносимость.



Эволюционные улучшения

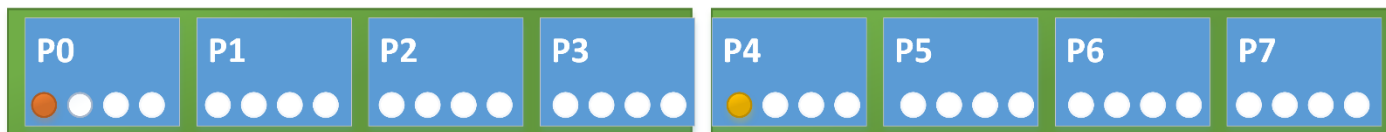
- В стандарте много эволюционных улучшений – поддержка новых версий языков программирования (в первую очередь, последних версий C++ (C++ 17, C++ 20 ожидается), Fortran 2008)
- Дополнение существующих конструкций в соответствии с потребностями программистов (пример – inout для depend, taskyield, collapse для вложенных циклов, собственные операторы для reduce и др.)
- Адаптация под существующие архитектурные особенности современных вычислительных систем (например – адаптация к тому, что современные системы в массе своей стали **ccNUMA** - Cache coherent Non-uniform memory access – «архитектура с несимметричным (неравномерным) доступом к памяти с когерентностью кешей» - **proc_bind** (варианты master, close, spread, также true и false)

Пример proc_bind

`export OMP_PROC_BIND= "spread" //максимально раскидать по процессорам`

`#pragma omp parallel proc_bind(spread) num_threads(N)`

spread 2



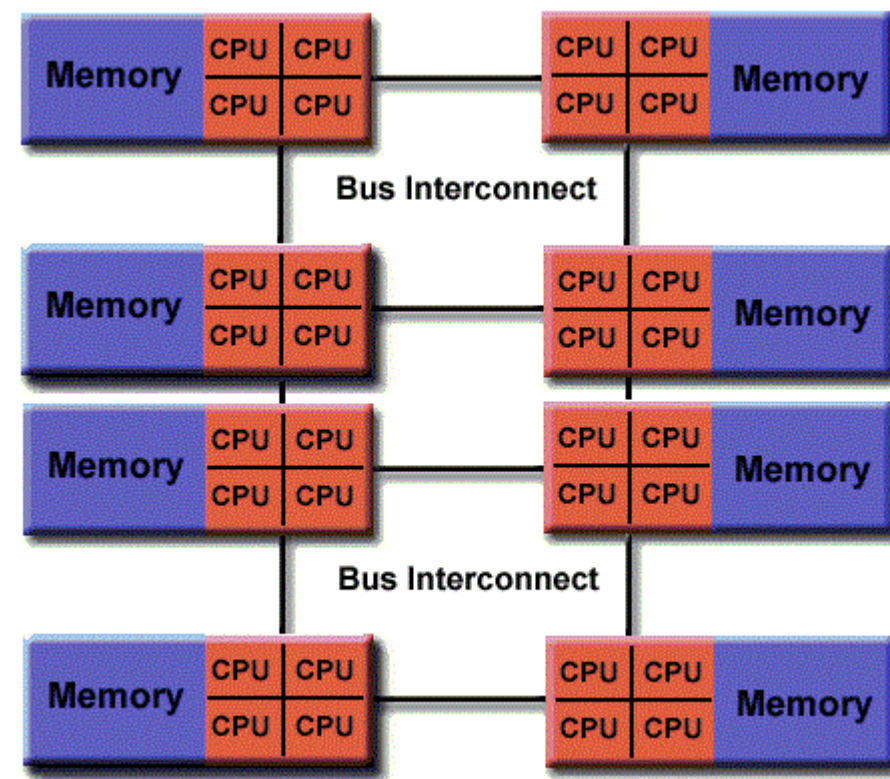
spread 4



spread 8



spread 16



Пример proc_bind

export OMP_PROC_BIND= "close"//поближе друг к другу

#pragma omp parallel proc_bind(close) num_threads(N)

close 2



close 4



close 8



Пример proc_bind

```
export OMP_PROC_BIND= "master" //все мастер-потоку  
#pragma omp parallel proc_bind(master) num_threads(N)
```

master 2



master 4



master 8



OMP_PROC_BIND

Глобальная переменная OMP_PROC_BIND (master, close, spread, а также false – все политики размещения игнорируются или true – тогда можно задавать через #pragma omp parallel proc_bind())

Посмотреть конфигурацию – lscpu, cat /proc/cpuinfo

(OS Linux)

Пример:

```
#pragma omp parallel proc_bind(spread)
```

```
{
```

```
    #pragma omp parallel proc_bind(close)
```

```
{
```

```
//какой-то код, приоритеты: внутренний блок – важна локальная общность данных,  
внешний – частота и наибольшая пропускная способность памяти
```

```
}
```

```
}
```

OMP_PLACES

Глобальная переменная OMP_PLACES (threads, cores, sockets)

Также есть возможность явной нумерации

```
export OMP_PLACES = threads
```

Каждый поток закреплён к своему потоку процессора

```
export OMP_PLACES = cores
```

Каждый поток закреплён к своему ядру процессора, может мигрировать между потоками своего ядра

```
export OMP_PLACES = sockets
```

Каждый поток закреплён к процессору в отдельном сокете, может мигрировать между ядрами и гиперпотоками своего сокета

OMP_PLACES - прямая нумерация

Прямое указание

```
export OMP_PLACES "{0,1,2,3},{4,5,6,7},{8,9,10,11}, ... {28,29,30,31} "
```

Или

```
export OMP_PLACES "{0:3}:8:4"
```

Формат шаблона:

```
{<границы>}:<число вхождений>:<шаг>
```

//оба варианта на слайде одинаковы

//через глобальные переменные – удобно конфигурировать

Пример улучшения - collapse

- Когда нужна конструкция – при наличии вложенных циклов, особенно если число итераций по внешнему циклу относительно небольшое, из-за чего страдает балансировка
- Что даёт – позволяет распределять между потоками итерации не только внешнего цикла, но и внутреннего (внутренних) (collapse(n))

```
#pragma omp parallel for collapse(2)
```

```
for (i = 0; i < imax; i++) {
```

```
    for (j = 0; j < jmax; j++)
```

```
        { // какой-то код
```

```
        }
```

```
}
```

Пример улучшения – ordered depend (sink)

```
#pragma omp for ordered (2)
```

```
for (int i = 0; i < M; i++)
```

```
    for (int j = 0; j < N; j++)
```

```
{
```

```
    a[i][j] = foo (i, j);
```

```
#pragma omp ordered depend (sink: i - 1, j) depend (sink: i, j - 1)
```

```
    b[i][j] = boo (a[i][j], b[i - 1][j], b[i][j - 1]);
```

```
#pragma omp ordered depend (source)
```

```
    bzz (a[i][j], b[i - 1][j]);
```

```
}
```

Только внешний цикл распараллеливается. ordered depend (sink) блокирует выполнение итераций i, j до тех пор, пока в итерациях $i-1$ и $j-1$ не достигнет следующей директивы ordered depend (source)

В данном случае проигнорирует, ибо к моменту i, j $i, j-1$ выполнена.

Принципиально новые возможности стандарта

- Стандарт активно адаптируется под новые архитектурные веяния, в том числе такие, как векторные инструкции, модель устройств (которая, в принципе, расширяется на GPU, Intel MIC и другие, выгрузка вычислений на ускорители называется [offload](#))
- В стандарте появляются новые парадигмы – так как задачи вместо секций и т.д., так как в целом акцент смещается с доработки существующего ПО (в тяжёлых случаях – legacy code) на разработку нового ПО
- Вопрос о распространении стандарта на distributed-memory – стоит, есть множество научных статей на эту тему, однако как будет реализован – вопрос пока открытый. Теоретически можно было бы из существующих конструкций реализовать через модель device – однако, много ограничений – в частности, несимметричность модели.

SIMD

По классификации Майка Флинна SIMD, введенной в 1966 г., это:

Single Instruction (Stream), Multiple Data – один поток команд, множественный поток данных

SIMD вычислительная система содержит управляющий модуль (контроллер) и процессорные элементы – модули обработки данных, у каждого процессорного элемента может быть собственная память для хранения данных

Примеры: ряд исторических архитектур суперкомпьютеров (ILLIAC IV, ICL Distributed Array Processor (DAP) и ряд других), большинство современных GPU

Также есть SIMD-расширения различных архитектур (используется принцип, но нет всех атрибутов полноценной SIMD вычислительной системы)

SIMD-расширения - основы

- Современные процессоры имеют в своём арсенале векторные инструкции – инструкции, работающие с одномерными массивами данных (которые называются векторами)
- Позволяют сделать одну операцию над большим количеством данных за один раз – тем самым многократно повышая производительность, при наличии соответствующих условий
- Повышают производительность в широком спектре задач (обработка мультимедийных данных, научные и инженерные вычисления, любые задачи, которые поддаются векторизации)...
- ...ценой увеличения сложности процессора, усложнения компилятора и необходимости, в ряде случаев, дополнительных действий со стороны программиста

SIMD-расширения в различных архитектурах

x86 и x86-64:

- MMX (Multimedia Extension) – 1997 г., первое расширение (только целочисленное, ряд ограничений)
- 3DNow! – 1998 г.(нынче устаревшее расширение, от компании AMD) – уже с поддержкой чисел с плавающей запятой
- SSE (Streaming SIMD Extensions) – первая версия в 1999 г., далее SSE2 (2000 г.), SSE3 (2004 г.), SSSE3 (2006 г.), SSE4 (2006 г.), SSE4a, SSE4.1, SSE4.2, SSE5 (предложен компанией AMD, не получил распространения)
- Advanced Vector Extensions (AVX) (2008 г.), AVX2 (2013 г.), AVX512 (2017 г.)

SIMD-расширения в различных архитектурах

ARM:

- SIMD, NEON, SVE, SVE2

MIPS:

- MDMX, MIPS-3D

SPARC:

- VIS, VIS2

RISC-V:

- P-extension
и другие.....

MMX

- 8 регистров по 64 бита (mm0 – mm7)
- каждый регистр - может содержать 2 32-битных, 4 16-битных или 8 8-битных целых числа (или одно значение в 64 бита, это времена когда основная архитектура была 32-битная)

Можно выделить группы инструкций:

- инструкции пересылки данных (позволяющих перемещать данные между регистрами MMX и целочисленными регистрами процессора или памятью)
- инструкции упаковки и распаковки данных (позволяющих преобразовывать элементы векторов в элементы с меньшей разрядностью или попарно объединять элементы с образованием элементов большей разрядности)

MMX – группы инструкций(продолжение)

- арифметические инструкции
- инструкции сравнения векторов с записью результата сравнения в виде битовой маски
- логические инструкции
- инструкции сдвиговые
- инструкции управления состоянием

SSE

- 16 дополнительных 128-битных регистров (в ранних версиях 8)

Название – от xmm0 до xmm15

- Управляющий регистр mxcsr
- Скалярные данные (просто одно значение на 128 бит) или векторизованные (упакованные) данные (scalar data или packed data)
- В каждом регистре могут содержаться 2 или 4 значения (64 или 32 бита) с плавающей запятой или 4, 8 или 16 целочисленных значений (на 32, 16 или 8 бит, соответственно)
- Расширен набор команд

Векторизация – пример ситуации

Пример – когда векторные инструкции дают выигрыш в производительности:

```
float A[ SIZE ] , B [ SIZE ] ;  
for ( int i = 0 ; i < SIZE ; i ++){  
    A[ i ] += B [ i ] ;  
}
```

Для архитектуры x86 требуется поддержка SSE или более новых версий, SSE - расширение, содержащее 128 битные регистры, в которые можно поместить 4 переменных типа float

В данном примере – нет информационных зависимостей, в результате чего появляется возможность выполнять итерации цикла одновременно

Выравнивание памяти (data alignment)

Для эффективной работы векторных инструкций рекомендуется, чтобы данные в памяти располагались выровненными по определённым адресам, кратным 16, 32 и т.д., в зависимости от разрядности соответствующих векторных инструкций.

Дело в том, что для обработки выровненных упакованных данных, например для SSE, извлечение 16-ти байтовых фрагментов данных будет выполняться за одну операцию.

Если же данные в памяти не выровнены, то процессору потребуется более одной операции для получения необходимого фрагмента данных, что замедлит выполнение.

Например, для SSE есть 2 типа инструкций, для выровненных и невыровненных упакованных данных. Вторые в среднем работают несколько более медленно.

Как выравнивать память?

Для ассемблера есть соответствующие команды (align 16 и т.п.)

В C11 появилась функция aligned_alloc (обычный malloc и realloc не гарантируют выравнивая данных по чему-то большему, чем sizeof (фундаментального типа данных, для которого вызываются)).

В свежей версии POSIX – posix_memalign (_POSIX_VERSION>=200112L), выделенную память можно освободить с помощью обычной функции free

У Microsoft есть _aligned_malloc, _aligned_free, _aligned_realloc

У компилятора Intel - _mm_malloc()

В GNU C - __attribute__ совместно с aligned (16) // например

```
int x __attribute__((aligned (16))) = 0;
```

```
char buffer[256] __attribute__((aligned (16)));
```

SIMD без OpenMP

- Самый трудоёмкий метод – прямые ассемблерные вставки или ассемблерный код
- Возможно получение хорошей производительности – средствами конкретного компилятора (в частности интринсики (intrinsics) — встроенные объекты компилятора) – например `__m128` – обозначение типа данных для SSE в gcc и ряде других компиляторов. Не входят в стандарт C/C++
- Низкоуровневые методы - можно добиться хорошей производительности, но требуется понимание целевой архитектуры, дополнительные трудозатраты для программиста и теряется переносимость
- Использование библиотек, например IPP (Intel Integrated Performance Primitives) – с ограничением этих библиотек
- Прямая поддержка векторизации компилятором

Диалекты языков C/C++

Intrinsics (интринсики SIMD) – расширения языков C/C++ для доступа к векторным инструкциям без ассемблерных вставок.

Для этого, подключаются библиотеки и даются директивы компилятору, что-то вроде:

```
#pragma GCC target("avx")  
#pragma GCC optimize("O3") // или ключами при компиляции  
#include<x86intrin.h>  
#include<bits/stdc++.h>
```

Далее используются соответствующие типы данных, например `_m128`, для 128 бит или `_m256`, для 256 бит. По умолчанию целочисленные, если вещественные двойной точности, то `d` - `_m256d` и так далее (в соответствии с типом инструкций)

Диалекты языков C/C++

Перед использованием можно также проверить поддержку инструкций CPU, например:

```
__builtin_cpu_supports("sse");
```

Пусть есть выровненные массивы a, b и c типа double

Далее, как вариант (тут суммируются элементы массивов)

```
for (int i = 0; i < 100; i +=4){  
    _m256d x = _mm256_loadu_pd(&a[i]);  
    _m256d y = _mm256_loadu_pd(&b[i]);  
    _m256d z = _mm256_add_pd(x, y);  
    _mm256_storeu_pd(&c[i], z);  
}
```

Векторизация - OpenMP SIMD

Для того, чтобы сообщить компилятору, что можно использовать векторные инструкции при распараллеливании того или иного цикла, используется конструкция `simd`

Формальный синтаксис:

```
#pragma omp simd [clause[,] clause] ...],
```

Может применяться с `for` (`#pragma omp for simd`)

Возможные дополнения:

- `aligned // aligned([ptr] : [alignment], ...)` – сообщаем что указатель выровнен по памяти (дополнительно указывается по какой разрядности)
- `reduction //` всё как и для обычных циклов

Векторизация - OpenMP SIMD - продолжение

Часто встречающиеся дополнения `omp simd`, продолжение:

- `safelen // safelen([value])`, интересная опция, связанная с информационными зависимостями, максимальное безопасное расстояние между итерациями при распараллеливании
- `simdlen` - желаемое число итераций цикла, подлежащее одновременному выполнению
- `collapse` // как и для обычных циклов
- `linear // linear([variable] : [step], ...)` – увеличение с индексом (с шагом)
- `private/lastprivate`

Пример

```
float A[ SIZE ] , B [ SIZE ] ; //Лучше конечно выровнять по памяти
```

```
#pragma omp simd
```

```
for ( int i = 0 ; i < SIZE ; i ++){
```

```
    A[ i ] += B [ i ] ;
```

```
} // выполняется на одном потоке, но с использованием векторных  
инструкций, возможна и комбинация:
```

```
float A[ SIZE ] , B [ SIZE ] ;
```

```
#pragma omp parallel for simd
```

```
for ( int i = 0 ; i < SIZE ; i ++){
```

```
    A[ i ] += B [ i ] ;
```

```
} // выполняется параллельно, с использованием ещё и векторных  
инструкций
```

Пример safelen

// по *i* *антизависимость* по данным, небезопасно
распараллеливать фрагментами больше 4

```
#pragma omp simd safelen(4)
```

```
{
```

```
  for (i=0; i<(N-4); i++) {
```

```
    a[i] = a[i+4] + b[i] * c[i]; //для  $a[i] = a[i-4] + \dots$  тоже будет работать  
    корректно, при пересчёте границ цикла
```

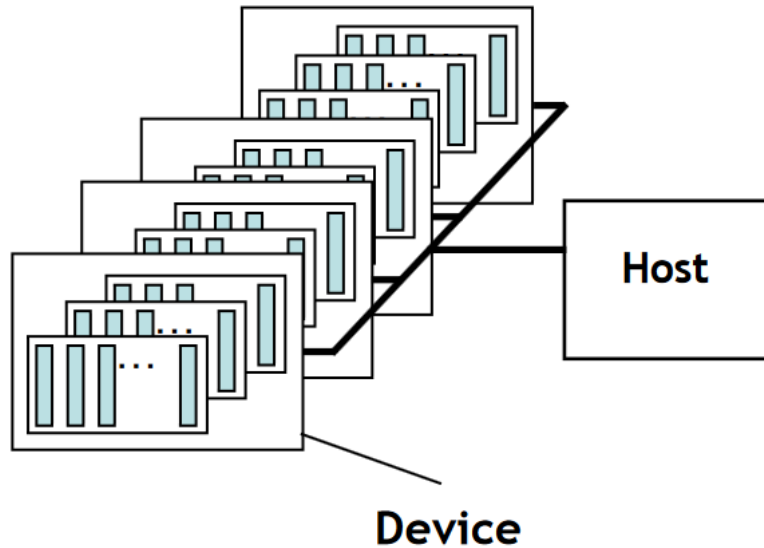
```
  }
```

```
}
```

Краткий обзор модели устройств в OpenMP (OpenMP device)

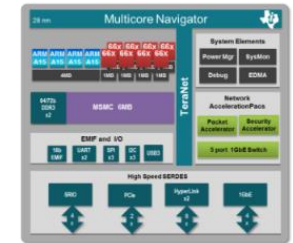
Примечание – возможность появилась в стандарте относительно недавно, всегда смотрите документацию на компилятор и на сами устройства для выяснения особенностей реализации.

Модель host - device

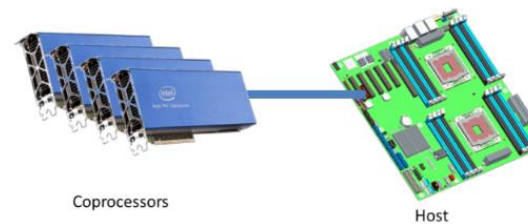


Device Model

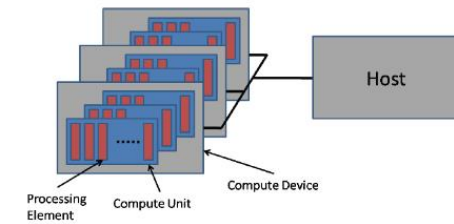
- OpenMP supports heterogeneous systems
- Device model:
 - One host device and
 - One or more target devices



Heterogeneous SoC



Host and Co-processors



Host and GPUs

Что подразумевается под устройством?

Устройства, в общем случае, могут быть:

- CPU (central processing unit, например, Intel MIC (Many Integrated Core – торговая марка Xeon Phi или вообще иная архитектура)
- GPU (graphics processing unit)
- FPGA (field-programmable gate array, ПЛИС)
- иные устройства

Также некоторые крупные компании для обозначения всех подобных ускорителей используют общий термин XPU.

Для чего чаще всего используется?

- Скалярные операции
- Векторные операции
- Матричные операции
- Обработка потоковых данных
- Пространственные вычисления (Spatial computing – «машина сохраняет и манипулирует ссылками на реальные объекты и пространства»)

Логика работы с устройствами

Настраиваем среду (самое сложное)

Определяем, сколько всего устройств есть. Функция

```
int omp_get_num_devices(void)
```

// также для выбора конфигурации и при работе есть ряд других функций и глобальных переменных

- void omp_set_default_device(int dev_num)
- int omp_get_default_device(void)
- int omp_get_num_teams(void) // модель предусматривает разделение исполнителей на группы
- int omp_get_team_num(void)
- int omp_is_initial_device(void)
- int omp_get_initial_device(void)
- Control default device through OMP_DEFAULT_DEVICE
- Control offloading behaviour OMP_TARGET_OFFLOAD (MANDATORY | DISABLED | DEFAULT)

... и другие (зависит от версии стандарта)

Логика работы с устройствами

Нам нужно определиться, какие функции будут использоваться на устройстве и какие области памяти откуда куда будут *мапироваться (data mapping)*.

Для функций используем конструкцию

```
#pragma omp declare target
```

//тут обычно какие-то функции, которые должны быть определены как на устройстве, так и на управляющем хосте, библиотеки и т.д.

```
#pragma omp end declare target
```

Пример:

```
#pragma omp declare target
```

```
void operate_on_device_initial(int* srcA, int* srcB, int* result); //иногда требуется extern....
```

```
void operate_on_device_final(int* srcA, int* srcB, int* result);
```

```
#pragma omp end declare target
```

Логика работы с устройствами

В какой-то момент требуется запустить мастер-поток на устройстве — это делается при помощи

```
#pragma omp target
```

```
{ //некий блок кода}
```

Работает с:

If

device(номер устройства)

private (список), firstprivate (список)...

nowait

data

defaultmap (dependence-type : list)

map (map-type: list[0:N])

depend (dependence-type : list)

update

и другими

На устройстве могут быть «команды потоков»
(teams & streams)

```
#pragma omp teams [clauses] //внутри target
```

```
    structured-block
```

```
clause:
```

```
num_teams(integer-expression)
```

```
thread_limit(integer-expression)
```

```
default(shared | none)
```

```
private(list)
```

```
firstprivate(list)
```

```
shared(list)
```

```
reduction(reduction-identifier : list)
```

Директива distribute

#pragma omp distribute [clauses] //внутри target, на устройстве
for-loops //некий цикл, итерации которого будут раскиданы
между мастер-потоками в командах

clause:

private(list)

firstprivate(list)

lastprivate(list)

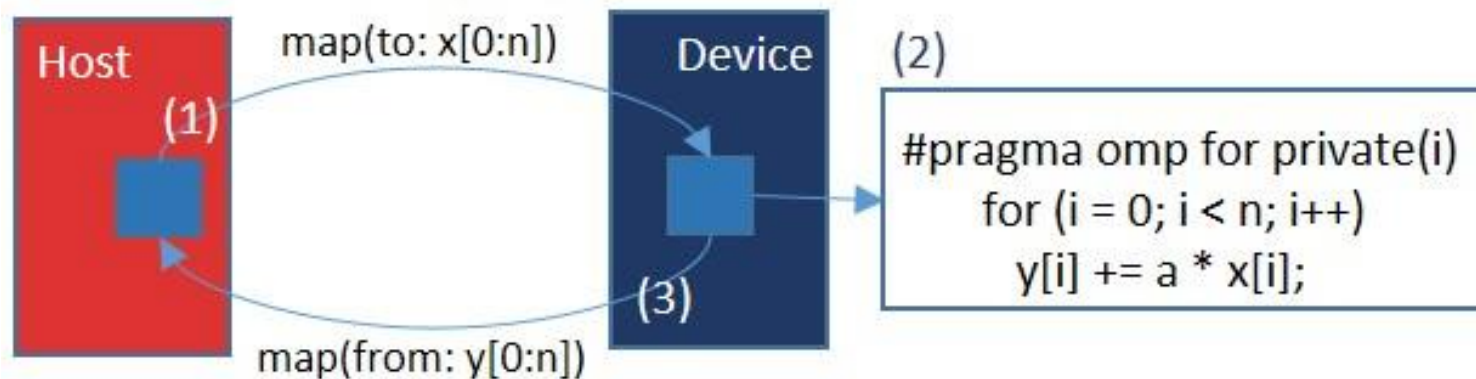
collapse(n)

dist_schedule(kind[, chunk_size])

Логика работы с устройствами

- target data – данные из host устройств мапируются в устройство для расширения региона ассоциации
- target update – синхронизируем буферы
- map (map-type: list[0:N]) – тип может быть to, from, tofrom, alloc, release, delete

Пример логики использования:



Логика работы с устройствами

На устройстве мастер-поток может создавать свои параллельные области, использовать векторные инструкции, команды и т.д. – относительно архитектурно-зависимо, однако полностью особенности архитектуры могут не копироваться

Синхронизация данных по мере надобности, выполнение вычислений, согласно архитектуре устройства

Завершение работы с устройством

Пример с target

```
#include <stdio.h>
#ifdef _OPENMP
#include <omp.h>
#endif
int main()
{
    int num_devices = omp_get_num_devices(); // также есть переменная окружения OMP_DEFAULT_DEVICE
    printf("Number of available devices %d\n", num_devices);
    #pragma omp target
    {
        if (omp_is_initial_device()) {
            printf("Running on host\n");
        } else {
            int nteams= omp_get_num_teams();
            int nthreads= omp_get_num_threads();
            printf("Running on device with %d teams in total and %d threads in each team\n",nteam,nthreads);
        }
    }
}
```

Пример с teams

```
float dotprod(float B[], float C[], int N)
{
    float sum0 = 0.0, sum1 = 0.0;
    #pragma omp target map(to: B[:N], C[:N])
    #pragma omp teams num_teams(2)
    {
        if (omp_get_team_num() == 0)
        {
            #pragma omp parallel for reduction(+:sum0)
            for (int i=0; i<N/2; i++)
                sum0 += B[i] * C[i];
        } else if (omp_get_team_num() == 1) {
            #pragma omp parallel for reduction(+:sum1)
            for (int i=N/2; i<N; i++)
                sum1 += B[i] * C[i];
        }
    }
    return sum0 + sum1;
}
```

Умножение векторов

```
extern void func_on_host();
extern void output_on_host(float *,int);
#pragma omp declare target
extern void init_on_device(float *,float *,int);
void vec_mult(float *p, int N, int dev)
{
    float *v1, *v2; int i;
    #pragma omp task shared(v1, v2) depend(out: v1, v2)
    #pragma omp target device(dev) map(v1, v2)
    {
        v1=malloc(N*sizeof(float)); v2=malloc(N*sizeof(float)); init_on_device(v1,v2,N);
    }
    func (); // Тут считается что-то другое
    #pragma omp task shared(v1, v2, p) depend(in: v1, v2)
    #pragma omp target device(dev) map(to: v1, v2) map(from: p[0:N])
    {
        #pragma omp parallel for
        for (i=0; i<N; i++) p[i] = v1[i] * v2[i];
        free(v1); free(v2);
    }
    #pragma omp taskwait
    output(p, N);
}
```